Linux Basics for Pentesters — A Hands-on Start

A practical introduction to the Linux command line, written for the pentester who is brand new to the operating system. Every concept is built around tools you will actually use during an assessment: navigating the filesystem, running commands as the right user, finding files, chaining commands with pipes, and scripting your first recon loop.

This walkthrough is written for a student who has used a computer but has never touched a Linux terminal before. Work through each step on your analyst box (Kali, Parrot, or any Debian based distro), compare what you get with the expected output, and if something looks wrong check the "Likely errors and fixes" block for that step before you ask for help.

The guide gets lighter as you go: the first steps explain every letter of every command, and the later steps expect you to remember the pattern and apply it yourself.

What this guide covers

By the end you will be able to sit at a Linux terminal and carry out the everyday tasks every assessment needs:

- understand the shell prompt and how commands, arguments and flags fit together;
- move around the filesystem and inspect files with `pwd`, `cd`, `ls`, `cat`, `less`, `file` and `head`;
- understand users, groups and file permissions, and know when to reach for `sudo`;
- find things fast with `grep`, `find` and `locate`;
- join tools together with pipes and redirection so one command feeds the next;
- write a short bash loop to repeat a scan across many hosts or ports;
- make a small shell script and run it as an executable;
- check your work and stay safe on a real target.

You will not become a scripting expert in one session, but you will have the vocabulary to read and run the commands in every later lab guide, and the confidence to look a tool up without panicking.

Deliberately out of scope: kernel building, package compilation from source, hardening, and anything that needs more than one terminal at once. This is the floor you stand on before you specialise, not the ceiling.

Before you start

You need a Linux box with a terminal. Any Debian based pentesting distro works: Kali Linux, Parrot Security OS, or a plain Ubuntu install with a few tools added.

1. Boot your Linux machine, log in, and open a terminal emulator. On Kali the terminal is the dark square icon pinned to the top bar. On a headless box you are already at a shell after login.

Likely errors and fixes: - You see a graphical desktop but no terminal icon. Fix: press `Ctrl` + `Alt` + `T` on most desktops, or search the applications menu for "Terminal", "Konsole" or "xterm". - The screen shows `login:` and nothing else. Fix: type your username, press Enter, type your password and press Enter. You are now at a shell prompt, which is exactly where you need to be.

1. Confirm you are really in a bash shell:

```
bash echo $SHELL
```

What this does: prints the full path of your login shell. `$SHELL` is a variable (more on those later) that holds your default shell program.

Breaking the command down: - `echo` prints whatever text you give it to the screen. - `$SHELL` is the *value* of a variable named `SHELL`; the `$` tells bash "look up the value, not the word SHELL". - The expected answer is `/bin/bash`. If you see `/bin/zsh` or something else, the guide still works, but a few command names may differ.

Likely errors and fixes: - Nothing prints, or the shell drops into a menu you did not expect. Fix: type `bash` and press Enter to start a bash shell explicitly, then try again.

1. Make sure you have the core tools this guide uses. On Kali and Parrot they are preinstalled; on a bare Ubuntu box install them once:

```
bash sudo apt-get update sudo apt-get install -y file grep findutils locate net-tools
```

What this does: refreshes the package list (`update`) and then installs (`install`) the tools we use later. The `-y` answers "yes" to the prompt automatically.

Likely errors and fixes: - `sudo: command not found`. Fix: you are not on a Debian family distro (that happens on Alpine or Arch). Either switch to a Kali/Parrot box or ask which package manager your distro uses and adapt these two lines. - `Unable to locate package locate`. Fix: some distros split `locate` into `mlocate` or `plocate`. Try `sudo apt-get install -y mlocate`, or simply skip it for now, it is optional.

Step 1 — Read the shell prompt

The first thing you see is not noise; it is a signpost telling you who you are and where you are. A typical prompt on a pentesting box looks like this:

```
kali@kali:~$
```

Breaking the prompt down, left to right: - `kali` before the `@` is your username. - `kali` after the `@` is the machine (host) name. - `~` is your current directory; `~` is shorthand for your home folder, usually `/home/kali`. - `$` means "you are an ordinary user". If this were `#` instead, you would be root, the all powerful administrator account.

When a guide tells you to "type a command", it means type the text that appears *after* the prompt and press Enter. You never type the prompt itself.

Check your understanding: predict what your own prompt says about your username, machine and current folder before moving on. If any part puzzles you, run the command in the next step to confirm.

Step 2 — Navigate the filesystem

Pentesting means spending most of your day moving between directories: dropping into `/etc` to read config, jumping to `/usr/share/wordlists` to grab a wordlist, and coming home to your working folder to save notes. Three commands cover almost all of it.

Step 2.1 — Where am I? `pwd`

```
pwd
```

What this does: prints the full path of your current directory. `pwd` stands for "print working directory".

Breaking the command down: - `pwd` is the whole command; there are no flags or arguments. - The output is an absolute path starting with `/`, for example `/home/kali`. An absolute path describes a location from the root of the filesystem, so it works no matter where you are.

Likely errors and fixes: - You expected `/home/kali` but got `/` or `/root`. Fix: that just means you opened the terminal in a different place, or you are logged in as root. Run `cd ~` to jump back to your home folder, then re-run `pwd`.

Step 2.2 — What is here? `ls`

```
ls
```

What this does: lists the names of files and folders in the current directory. On its own it prints a short, bare list.

The two flags you will use constantly:

```
ls -l
ls -la
```

What these do: `-l` gives the long format (permissions, owner, size, date), and `-a` adds hidden files, the ones whose names begin with a dot. Together, `-la` is "give me everything, in detail", which is the default you will want on almost every target.

Breaking `ls -la` down: - `-l` = long listing. - `-a` = all files, including hidden ones. - `ls -la` and `ls -al` are the same command; the order of single letter flags does not matter.

You can also list a specific folder without moving into it:

```
ls -la /etc
```

Likely errors and fixes: - `ls: cannot access '/etc': No such file or directory`. Fix: you have misspelled the path. Absolute paths start with `/`, so check you typed `/etc` and not `etc` (which would mean a folder called `etc` inside your current directory). - The list is too wide and wraps awkwardly. Fix: use `ls -l` alone, which prints one entry per line.

Step 2.3 — Move around `cd`

```
cd /etc
pwd
ls | head
```

What this does: `cd` changes directory to `/etc`, `pwd` confirms you are there, and `ls | head` shows only the first ten entries so the screen is not flooded. The `|` (pipe) and `head` are explained fully in a later step; for now read `ls | head` as "list, but show me just the top".

Breaking `cd /etc` down: - `cd` = change directory. - `/etc` = where you want to go; an absolute path.

The special directories you will meet constantly:

```
cd ~        # home folder
cd ..       # one level up
cd .        # current folder (rarely used, but explains the dots)
cd -        # the folder you were in before this one
```

Likely errors and fixes: - `cd: no such file or directory: /etc`. Fix: check the spelling and the leading `/`. `cd /etc` (from root) is not the same as `cd etc` (from wherever you are). - Typing `cd` with no argument at all does nothing visible. Fix: that is expected; `cd` with no argument returns you to your home folder. Run `pwd` to see it happened.

Step 3 — Look inside files

On a target you spend your time reading: config files for credentials, `/etc/passwd` for user names, log lines for failed logins, source code for bugs. These four readers cover nearly everything.

Step 3.1 — Dump a file to the screen `cat`

```
cat /etc/hostname
cat /etc/os-release
```

What this does: prints the entire contents of a file, one after another. `cat` is short for "concatenate", which reflects its real job of joining files, but in practice everyone uses it to just dump a small file.

Breaking `cat /etc/hostname` down: - `cat` = read and print the file. - `/etc/hostname` = the file to read.

Use `cat` for *small* files. For anything longer than one screen it is the wrong tool.

Likely errors and fixes: - `cat: /etc/hostname: No such file or directory`. Fix: the file does not exist here. Check your path, or use `ls /etc/host*` to see what is actually there. - The output whizzes past and you only see the last few lines. Fix: you should have used `less` instead (next step). Do not panic, re-run with `less`.

Step 3.2 — Read a long file a screen at a time `less`

```
less /etc/passwd
```

What this does: opens a pager, showing one screen of the file at a time. Inside `less`, use the arrow keys or `Page Up` / `Page Down` to scroll, and press `q` to quit back to the prompt.

Likely errors and fixes: - You are stuck inside the file and cannot type commands. Fix: you are not "stuck"; you are in `less`. Press `q` to leave it. - `less` runs but the file is empty. Fix: `/etc/passwd` is never empty, so you are reading the wrong path. Confirm with `ls -l /etc/passwd`.

Step 3.3 — The first few lines `head`

```
head /etc/passwd
head -n 3 /etc/passwd
```

What this does: prints the first ten lines by default, or the first `n` lines when you pass `-n`. Use `head` to peek at a file without committing to reading all of it.

Likely errors and fixes: - You wanted the *end* of the file, not the start. Fix: use `tail` for the last lines, especially useful on logs (see Step 4.3 for a real use).

Step 3.4 — What kind of file is this? `file`

```
file /etc/passwd
file /bin/ls
```

What this does: inspects the *content* of a file and tells you what it really is, which is not the same as what its name suggests. This is a core pentesting habit: a file named `report.txt` could actually be an ELF executable or an image, and `file` will tell you.

Likely errors and fixes: - `file: cannot open`. Fix: the path is wrong or the file does not exist. Check the spelling and that you are in the right directory.

Check your understanding: without running anything, predict the output of `file /bin/bash` and `head -n 1 /etc/os-release`, then run them to see if you were right.

Step 4 — Users, permissions and `sudo`

Security on Linux is built on one simple rule: every file belongs to a user and a group, and every user has a set of permissions. Understanding this rule tells you instantly why a command is refused, and how a pentester abuses misconfigured permissions to move up.

Step 4.1 — Who am I? `whoami` and `id`

```
whoami
id
```

What these do: `whoami` prints your username; `id` prints your user ID (UID), your primary group, and every group you belong to.

Breaking `id` down: - The output starts like `uid=1000(kali) gid=1000(kali) groups=1000(kali),...` - `uid` is your user ID, `gid` your primary group, and `groups` lists all the groups you belong to (some users belong to several). - The numbers matter: `0` is root. If you ever see `uid=0`, you are the administrator and should be extra careful.

Step 4.2 — Read the permissions `ls -l`

Run this and look closely at the left hand column:

```
ls -l /etc/passwd /etc/shadow
```

You will see something like:

```
-rw-r--r-- 1 root root 1234 Sep  9 10:00 /etc/passwd
-rw-r----- 1 root shadow 1000 Sep  9 10:00 /etc/shadow
```

The first ten characters are the permission string. Breaking `-rw-r--r--` down into its three groups:

- `-` the file type (`-` ordinary file, `d` directory, `l` link).
- `rw-` the *owner's* permissions: read, write, no execute.
- `r--` the *group's* permissions: read only.
- `r--` *everyone else's* permissions: read only.

The three letters repeat in each group: `r` read, `w` write, `x` execute (a dash means that permission is absent).

Now look at `/etc/shadow`. Its group permission is `r--`, meaning members of the `shadow` group can read it, but normal users cannot. Your `kali` user is not in that group, so you cannot read password hashes directly. That is not an accident; it is the security model working. A pentester who finds a way to read `/etc/shadow` has found a serious misconfiguration.

Likely errors and fixes: - `ls -l` shows permissions but you cannot read a file anyway. Fix: reading needs at least `r` on the file for your user or group. `r-x` on a *directory* allows entering and listing it, but not renaming or deleting files inside.

Step 4.3 — Become the administrator `sudo`

Some files are owned by root and only root may read or change them. Rather than logging in as root, you use `sudo` in front of a command to run just that one command as the administrator.

```
sudo cat /etc/shadow
```

What this does: prompts for *your* password (not root's), and if you are in the `sudo` group, runs `cat /etc/shadow` with root privileges, letting you read the password hashes.

Breaking the command down: - `sudo` = "superuser do", run the rest as root. - `cat /etc/shadow` = the command to run as root.

Likely errors and fixes: - `kali is not in the sudoers file. This incident will be reported.` Fix: your account is not permitted to use `sudo`. On your own test box, log in as root (or ask whoever set the box up) and add your user to the `sudo` group. - `Sorry, try again.` after typing your password. Fix: you typed the wrong password, or you are not a sudo user. Check with `id` that `sudo` appears in your groups. - `sudo: command not found`. Fix: the box may not have sudo installed, or you are on a non Debian distro (see Before you start).

When to use sudo, when not to: use `sudo` only when a command is refused for lack of permission. Running every command as root is a bad habit: it hides which files you actually need elevated access to, and on a real engagement you do not have root on the target anyway. Practice working as an ordinary user and reaching for `sudo` only when the error demands it.

Step 5 — Find things fast

Two of the most repeated pentesting actions are "find the file" and "find the line". These four tools cover both.

Step 5.1 — Search inside files `grep`

```
grep -i password /etc/services
```

What this does: prints every line in `/etc/services` that contains the word `password`, ignoring case (the `-i`). `grep` searches text, and it is the tool you will use to hunt for credentials, unusual services, and patterns in log files.

Useful flags you will rely on: - `-i` ignore case, so `Password`, `PASSWORD` and `password` all match. - `-v` invert, print lines that do *not* match (great for filtering noise). - `-n` show the line number of each match. - `-r` search recursively through a folder and its subfolders. - `-l` list only the names of files that contain a match, not the lines.

Recon use: search a whole config folder for secrets in one go:

```
grep -ril password /etc/ 2>/dev/null
```

What this does: `-r` recurses, `-i` ignores case, `-l` lists filenames only. The `2>/dev/null` quietly discards the "permission denied" errors from folders you cannot read, so the screen shows only real results.

Likely errors and fixes: - `grep: /etc/services: No such file or directory`. Fix: check the path. - No output even though the word is definitely there. Fix: you forgot `-i` and the file uses different capitalisation, or the word is split across lines.

Step 5.2 — Find files by name `find`

```
find / -name "shadow" 2>/dev/null
```

What this does: walks the whole filesystem from `/` looking for any file or folder whose *name* is exactly `shadow`. The `2>/dev/null` hides permission errors again.

Useful patterns: - `find / -name "*.conf"` finds everything ending in `.conf`. - `find / -type f -name "*.sh"` restricts to ordinary files (`-type f`). - `find / -type d -name "admin"` finds directories named `admin`.

Recon use: locate every writable file owned by root, a classic privilege escalation check:

```
find / -perm -4000 -type f 2>/dev/null
```

What this does: finds files with the setuid bit set (the `-perm -4000`), a program that runs with the owner's privileges whoever launches it. Setuid binaries are worth a long look on an engagement.

Likely errors and fixes: - The search is slow and seems to hang. Fix: `find /` scans the whole disk; that is normal. Narrow the start point, for example `find /etc -name "*.conf"`, and add `2>/dev/null` so permission errors do not flood you. - `find: paths must precede expression`. Fix: you quoted something wrongly, usually a `*` that the shell expanded. Put the pattern in double quotes, `-name "*.conf"`, not bare `*.conf`.

Step 5.3 — Find a file instantly `locate`

```
sudo updatedb
locate wordlist
```

What these do: `updatedb` builds a database of every file on the system (needs sudo because it must read all folders), and `locate` then searches that database by name. It is far faster than `find /`, but the database only knows about files that existed when `updatedb` last ran.

Recon use: find the wordlists bundled with your distro:

```
locate rockyou.txt
```

Likely errors and fixes: - `locate: command not found`. Fix: the `locate`/`mlocate`/`plocate` package is not installed (see Before you start). - `locate: can not stat ()`. Fix: run `sudo updatedb` first; the database is empty on a fresh install. - `locate` returns nothing for a file you know exists. Fix: the file was created after the last `updatedb`. Re-run `sudo updatedb`.

Step 5.4 — One page at a time `less` + search

```
less /var/log/syslog
```

Inside `less`, press `/` then type a search term and Enter to jump to the first match, and `n` to go to the next one. This is how you skim a long log for one interesting event without reading every line.

Likely errors and fixes: - Pressing `/` seems to do nothing. Fix: you are not inside `less`; you are back at the shell. Run `less <file>` again, then try `/`. - `n` does nothing. Fix: there is only one match, or you never ran a search with `/`.

Step 6 — Join commands with pipes and redirection

Single commands are useful, but the power of the shell is joining them. A pipe (`|`) feeds the output of one command into the next; redirection (`>`) saves output to a file. Almost every recon command you ever read is a chain of these.

Step 6.1 — Pipe one command into the next `|`

```
ls /usr/share/wordlists | head
```

What this does: `ls` lists the wordlists folder, and `head` shows only the first ten lines of that list. Read the pipe as "and then feed that into".

The pipe is the single most important habit in shell use. It lets you take the raw output of a scan and run it through `grep`, `head`, `wc` or anything else without saving a file first.

Recon use: count the lines in a wordlist without opening it:

```
wc -l /usr/share/wordlists/rockyou.txt
```

What this does: `wc -l` counts lines. Reading it as "word count, lines", you now know how many passwords are in the wordlist, which matters when you estimate how long a brute force will take.

Likely errors and fixes: - `command not found` for the second part of the pipe. Fix: the second command is misspelled or not installed. Each side of the `|` is a full command, so it must exist on its own. - The output is still huge. Fix: add `| head` or `| less` at the very end to page it, or pipe into `grep` to filter before you look.

Step 6.2 — Save output to a file `>`

```
ls -la /etc > etc-listing.txt
cat etc-listing.txt | head
```

What this does: the `>` sends the output of `ls -la /etc` into a new file called `etc-listing.txt` instead of the screen. Nothing prints. The second command confirms the file exists by printing its first lines.

Recon use: keep a copy of a scan result for your report:

```
nmap -sV -p- 172.20.0.3 > scan-results.txt
```

Now your findings survive after the scan window closes.

Two redirection gotchas to remember: - `>` *overwrites* the file if it already exists. To *append* instead, use `>>`. - `2>` redirects error output. You have already seen `2>/dev/null`, which sends errors to a special sink that discards them. `1>` (or just `>`) is normal output, `2>` is errors.

Likely errors and fixes: - The command prints nothing and you expected to see output. Fix: that is normal; the output went into the file. Check it with `cat <file>`. - You overwrote a file you wanted to keep. Fix: there is no undo in the shell. From now on prefer `>>` when you are not certain a file is disposable, and use `>` only for files you are happy to replace.

Step 7 — The shell variables and your PATH

Two more concepts and you can read most scripts on any pentesting box.

Step 7.1 — Shell variables

A variable is a named box holding a value. You met one already: `$SHELL` held the path to your shell. Set and read your own like this:

```
TARGET=172.20.0.3
echo "Scanning $TARGET"
```

What this does: stores the IP in a variable called `TARGET`, then prints `Scanning 172.20.0.3`. The `$` in the second line looks up the value.

Likely errors and fixes: - `TARGET: command not found`. Fix: there must be *no spaces* around the `=` in `TARGET=172.20.0.3`. `TARGET = 172.20.0.3` is parsed as "run a command called TARGET", which fails. - The variable is empty in your next terminal window. Fix: variables set at the prompt are temporary; they vanish when the shell closes. For permanent ones you would edit a config file, which is out of scope here.

Step 7.2 — The PATH

When you type `nmap`, the shell does not search the whole disk; it checks each folder listed in the `PATH` variable. Inspect yours:

```
echo $PATH
```

You will see a `:` separated list like `/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:...`. The shell tries each folder in turn until it finds an executable called `nmap`.

Practical consequence: if you download a tool to a folder not on your PATH, typing its name fails with `command not found`. You run it by giving its path instead:

```
./mytool.sh
```

The `./` means "right here, in the current folder". The tool name on its own would fail because the current folder is usually not on the PATH.

Likely errors and fixes: - `./mytool.sh: Permission denied`. Fix: the file exists but is not executable (see Step 8.2). Run `chmod +x mytool.sh` and try again. - `./mytool.sh: No such file or directory` even though `ls` shows it. Fix: the file may be a script whose interpreter line points at a program that does not exist, or a binary for the wrong architecture. Run `file mytool.sh` to check.

Step 8 — Write and run a shell script

A scan that takes twenty manual commands becomes a single command once you put those commands in a script. This is where the guide's scaffolding starts to fall away: by now you should be following the *pattern*, not every letter.

Step 8.1 — Create a script file

Use a text editor to create a file. The beginners' editor is `nano`:

```
nano recon.sh
```

Inside nano, type these four lines:

```
#!/bin/bash
TARGET=172.20.0.3
echo "Scanning $TARGET"
ping -c 3 "$TARGET"
```

Then save and exit: press `Ctrl` + `O`, Enter to confirm the filename, then `Ctrl` + `X` to quit.

What each line does: - `#!/bin/bash` is the *shebang*: it tells the shell to run this file with `/bin/bash`, whatever shell you happen to be in. It must be the very first line, no leading spaces. - `TARGET=...` sets a variable. - `echo "Scanning $TARGET"` prints what we are about to do. - `ping -c 3 "$TARGET"` pings the target three times.

Likely errors and fixes: - `nano: command not found`. Fix: install it (`sudo apt-get install -y nano`) or use `vi`, the other common editor. - The shebang is not line one. Fix: everything breaks mysteriously. Delete and retype so `#!` is at position zero of the first line.

Step 8.2 — Make it executable and run it

```
chmod +x recon.sh
./recon.sh
```

What these do: `chmod +x` adds the execute permission (the `x` you learned about in Step 4.2), and `./recon.sh` runs the script from the current folder.

Confirm the permission change:

```
ls -l recon.sh
```

You should now see `-rwxr-xr-x` instead of `-rw-r--r--`, the added `x` marks being the execute bits.

Likely errors and fixes: - `./recon.sh: Permission denied`. Fix: you forgot `chmod +x recon.sh`. - You ran `bash recon.sh` instead of `./recon.sh` and it worked anyway. Fix: that is fine, `bash <file>` runs a script without needing the execute bit. It is a handy alternative, but `./` is the habit that shows you understand permissions.

Check your understanding: edit `recon.sh` to also save the ping output to a file with `>` and run it again. When it prints nothing to the screen, confirm the file was created with `ls -l` and read it with `cat`.

Step 9 — Loop a scan across hosts and ports

The pay-off of the whole guide: one line of bash that repeats a tool across many targets or ports. This is the scaffolding finally falling away, and you should now be reading these chains as sentences, not decoding letters.

Step 9.1 — A `for` loop

```
for ip in 172.20.0.3 172.20.0.4; do echo "Host $ip"; ping -c 1 "$ip" | head -1; done
```

What this does: for each IP in the list, it prints the host name and then one line of the ping reply. Read it as: "for each ip in this list, do: echo and ping, then done".

Breaking the loop down: - `for ip in ...` names a variable (`ip`) that takes each value in the list. - `; do` marks the start of the commands to repeat. - `echo "Host $ip"` labels the output so you can tell the hosts apart. - `ping -c 1 "$ip" | head -1` runs one ping and keeps just the first line, so the loop stays tidy. - `; done` marks the end of the loop.

Likely errors and fixes: - `syntax error near unexpected token`. Fix: the spaces and semicolons are essential. Type `for ip in 172.20.0.3 172.20.0.4; do echo "Host $ip"; done` with the exact spacing, or put each part on its own line in a script. - It runs but prints nothing useful. Fix: the target may be down, or you are not on the right network. Confirm you can reach one host first with a plain `ping -c 1 <ip>`.

Step 9.2 — Loop a real tool: quick port scan

```
for port in 22 80 443 8080; do echo "Port $port"; timeout 2 bash -c "echo >/dev/tcp/172.20.0.3/$port" && echo "
```

What this does: for each port, it tries to open a TCP connection to the target. If the connection succeeds, the `&&` runs `echo " open"`; if it fails, nothing is printed for that port. The `timeout 2` stops each attempt after two seconds so a closed port cannot hang the loop.

Breaking it down: - `timeout 2` caps the next command at two seconds. - `bash -c "echo >/dev/tcp/<ip>/<port>"` is a shell trick: writing to `/dev/tcp/host/port` attempts a TCP connection. It is a quick way to test a single port without a scanner. - `&&` runs the next command only if the previous one succeeded, so " open" prints exactly when the port answers.

Likely errors and fixes: - Nothing prints at all. Fix: all ports may be closed, or the host is unreachable. First confirm with `ping -c 1 <ip>`, then test one known-open port manually. - The

loop takes a long time. Fix: the `timeout 2` per closed port adds up. Use a shorter timeout (`timeout 1`) or a proper scanner like `nmap` for a real assessment; this loop is for learning, not for scanning a whole range.

> *This loop is a teaching device. On a real engagement you would use `nmap` for a full port scan. The point of the loop is not to replace `nmap`, it is to make the idea of "repeat a tool across a range" automatic, because you will write such loops constantly with other tools.*

Step 10 — Check your work and stay safe

The final habit is not a command but a discipline. Before you run anything on a machine you do not own, make sure you are allowed to be there.

Step 10.1 — Verify every command you type

When a guide (or your memory) gives you a command, read it twice before you press Enter, especially when it involves `sudo`, `>` or a loop. Ask three questions:

- What does each part do, and do I understand the whole thing?
- Where is this writing? A `>` will overwrite a file, a loop can hammer a host.
- Is this the right target? Check the hostname or IP before you act, never after.

Confirm where you are before running anything destructive:

```
whoami
pwd
hostname
```

What this does: reminds you who you are, where you are, and which machine you are on. When an engagement goes wrong, these three lines of context save you.

Likely errors and fixes: - You ran a command on the wrong host. Fix: slow down. `hostname` first, then act. There is no "undo" on a live target.

Step 10.2 — The authority rule

Everything in this guide assumes you have permission to run it. On your own labs and training boxes, that is automatic. On a real target, permission is a written agreement, not an assumption.

The rule, stated plainly:

> *Only run these commands against systems you own or have written authorisation to test. Unauthorised scanning, access or modification is illegal in most jurisdictions and will get you into serious trouble. When you are practising, use isolated labs and deliberately vulnerable training targets, never live third-party systems.*

This is not a formality; it is the boundary between a professional assessment and a criminal offence. Make it your default.

Recap and what to practise next

You now have the floor skills every pentest runs on:

- **Navigate** with `pwd`, `cd`, `ls` and read files with `cat`, `less`, `head`, `tail`, `file`.
- **Understand access** with `whoami`, `id`, `ls -l` permissions and `sudo`.
- **Find things** with `grep`, `find`, `locate`.
- **Chain tools** with pipes `|` and redirection `>` / `>>`.

- **Automate** with a variable, a `for` loop and a `chmod +x` script.

The best next step is to practise each block against a deliberately vulnerable target. Work through a training lab (TryHackMe and Hack The Box both have beginner Linux rooms) and try to use `grep`, `find` and pipes instead of clicking around a file manager. The moment a command chain becomes faster than opening a folder in a GUI, you are working like a pentester.

Further guides in this set assume these skills and build on them: - **CPSA-GUIDE-01** (Network Discovery & Port Scan) assumes you can read a command, navigate the analyst box and save output with `>`. - **CPSA-GUIDE-02** (Enumeration) leans on `grep` to filter scan output. - **CPSA-GUIDE-05** (SMTP) uses `find` and `grep` to inspect mail and wordlists.

When you are ready to automate your recon for real, the next step after this guide is writing your own script that runs `nmap`, saves the output, greps the open ports and prints a tidy summary, which is exactly the loop from Step 9 scaled up.

Quick reference card

A one-screen summary of every command in this guide. Copy it into your notes folder and keep it open until the syntax is automatic.

Task Command Notes Where am I	`pwd`	prints current folder
What is here	`ls -la`	long, all files incl. hidden
Change folder	`cd /path`	`cd ..` goes up one
Go home	`cd ~`	or just `cd`
Dump a file	`cat file`	small files only
Read a long file	`less file`	`q` quits, `/` searches
First / last lines	`head file` / `tail file`	`-n 5` sets count
What type is it	`file file`	inspects real content
Who am I	`whoami` / `id`	`uid=0` is root
Read as admin	`sudo command`	your password, not root's
Find text	`grep -i pattern file`	`-r` recurse, `-v` invert
Find a file	`find / -name "name"`	`2>/dev/null` hides errors
Find instantly	`locate name`	run `sudo updatedb` first
Chain commands	`cmd1 | cmd2`	feed one output into next
Save output	`cmd > file`	`>>` appends instead
Count lines	`wc -l file`	useful on wordlists
Run a script	`chmod +x s.sh && ./s.sh`	`./` means current folder
Loop a range	`for x in a b c; do cmd; done`	repeat cmd per value
Run as root only sometimes	`sudo`	prefer least privilege